

COURSE PROJECT
EE1003
SCIENTIFIC PROGRAMMING

24 Hour Digital Clock

CONTENTS

1	Introduction	3
2	Hardware Implementation	3
3	Software Implementation	3
4	User Interface	8

1 INTRODUCTION

I have built a 24 hour clock, with 2 modes(normal mode, set mode), running with help of incremental logic.

2 HARDWARE IMPLEMENTATION

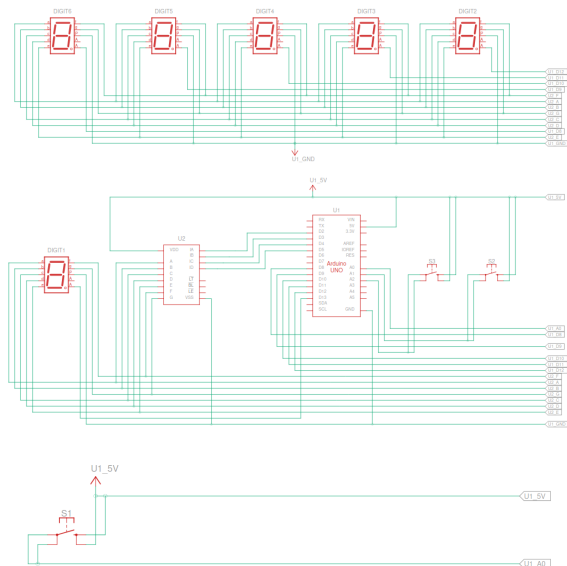


Fig. 0: Circuit diagram

The circuit diagram of the clock is given by 0. There are six 7-segment displays(DIGIT1 - DIGIT6), three buttons (S1 - S3), one 7447 BCD to 7-Segment decoder, and one Arduino.

As we are limited by the number of pins on the Arduino, we use the concept of multiplexing. The process will be explained in section 3

3 SOFTWARE IMPLEMENTATION

```

1 #include <avr/io.h>
2 #include <util/delay.h>
3
4 #define byte uint8_t
5
6
7 byte incUn(byte led);
8 byte incTenMin(byte led);
9 byte incTenHr(byte led);
10
11 int main(){
12     DDRB |= 0x3F;

```

```

13  DDRD |= 0x3C;
14  DDRC |= 0x07;
15
16  byte L1 = 0x00;
17  byte L2 = 0x00;
18  byte L3 = 0x00;
19  byte L4 = 0x14;
20  byte L5 = 0x0C;
21  byte L6 = 0x08;
22  byte flip = 0x00;
23  int count = 84;
24
25  byte prevState = 0x00;
26  byte buttonSelect = 0x00;
27
28  while(1){
29
30      if(!(L1 ^ 0x24)){
31
32          if(!(L2 ^ 0x14)){
33
34              if(!(L3 ^ 0x24)){
35
36                  if(!(L4 ^ 0x14)){
37
38                      if(!(L5 ^ 0x24)){
39
40                          L6 = incTenHr(L6);
41                      }
42
43                      else if(!(L6 ^ 0x08)){
44                          if(!(L5 ^ 0x0C)) {
45                              L5 = 0x24;
46                              L6 = 0x00;
47                          }
48                      }
49                      L5 = incUn(L5);
50                  }
51                  L4 = incTenMin(L4);
52              }
53              L3 = incUn(L3);
54          }
55          L2 = incTenMin(L2);
56      }
57      L1 = incUn(L1);
58
59
60      // PORTB |= 0xFF;
61      for(int i=1;i<count;i++){
62          byte buttons = PINC;
63
64          if((PINC & (1)) != (prevState & (1))){
65              if(PINC & (1)){

```

```

66         if(!(buttonSelect)){
67             buttonSelect = 0x01;
68         }
69         else{
70             buttonSelect = 0x00;
71         }
72     }
73 }
74 // prevState ^= (1);
75
76 if((PINC & (1<<1)) != (prevState & (1<<1))){
77     if((PINC & (1<<1)) && (buttonSelect)){
78         buttonSelect = buttonSelect<<1;
79         if(buttonSelect == 0x08) buttonSelect = 0x01;
80     }
81 }
82
83 if((PINC & (1<<2)) != (prevState & (1<<2)) && (buttonSelect)){
84     if((PINC & (1<<2))){
85         switch (buttonSelect)
86         {
87             case 0x01:
88                 L1 = 0x00;
89                 L2 = 0x00;
90                 break;
91
92             case 0x02:
93                 if(L3 == 0x24){
94                     L4 = incTenMin(L4);
95                 }
96                 L3 = incUn(L3);
97                 break;
98
99             case 0x04:
100                 if(!(L5 ^ 0x24)){
101
102                     L6 = incTenHr(L6);
103                 }
104
105                 else if(!(L6 ^ 0x08)){
106                     if(!(L5 ^ 0x0C)) {
107                         L5 = 0x24;
108                         L6 = 0x00;
109                     }
110                 }
111                 L5 = incUn(L5);
112             default:
113                 break;
114         }
115     }
116 }
117 // if(buttons & (1<<7))
118 prevState = buttons;

```

```

119     if(!(buttonSelect)){
120         PORTD = L1;
121         PORTB = 1;
122         _delay_ms(2);
123
124         PORTD = L2;
125         PORTB = (1<<1);
126         _delay_ms(2);
127
128         PORTD = L3;
129         PORTB = (1<<2);
130         _delay_ms(2);
131
132         PORTD = L4;
133         PORTB = (1<<3);
134         _delay_ms(2);
135
136         PORTD = L5;
137         PORTB = (1<<4);
138         _delay_ms(2);
139
140         PORTD = L6;
141         PORTB = (1<<5);
142         _delay_ms(2);
143     }
144     else{
145         if(!((buttonSelect & 0x01) & (i<count/2))){
146             PORTD = L1;
147             PORTB = 1;
148         }
149         _delay_ms(2);
150         if(!((buttonSelect & 0x01) & (i<count/2))){
151             PORTD = L2;
152             PORTB = (1<<1);
153         }
154         _delay_ms(2);
155         if(!((buttonSelect & 0x02)>>1 & (i<count/2))){
156             PORTD = L3;
157             PORTB = (1<<2);
158         }
159         _delay_ms(2);
160         if(!((buttonSelect & 0x02)>>1 & (i<count/2))){
161             PORTD = L4;
162             PORTB = (1<<3);
163         }
164         _delay_ms(2);
165         if(!((buttonSelect & 0x04)>>2 & (i<count/2))){
166             PORTD = L5;
167             PORTB = (1<<4);
168         }
169         _delay_ms(2);
170         if(!((buttonSelect & 0x04)>>2 & (i<count/2))){
171             PORTD = L6;

```

```

172         PORTB = (1<<5);
173     }
174     _delay_ms(2);
175 }
176 }
177 }
178 }
179
180 byte incUn(byte led1){
181     byte led = led1 >> 2;
182     byte D = led & 1;
183     byte C = (led & (1<<1))>>1;
184     byte B = (led & (1<<2))>>2;
185     byte A = (led & (1<<3))>>3;
186
187     byte W = (A&(C^1)&(D^1)) | (B&C&D);
188     byte X = ((B^1)&C&D) | (B&(C^1)) | (B&(D^1));
189     byte Y = ((A^1)&(C^1)&D) | (C&(D^1));
190     byte Z = D^1;
191
192     byte ledOut = Z | (Y<<1) | (X<<2) | (W<<3);
193     ledOut = ledOut << 2;
194     return ledOut;
195 }
196 }
197
198 byte incTenMin(byte led1){
199     byte led = led1>>2;
200
201     byte Z = led & 1;
202     byte Y = (led & (1<<1))>>1;
203     byte X = (led & (1<<2))>>2;
204
205     byte A = (Y&Z) | (X&(Y^1)&(Z^1));
206     byte B = ((X^1)&(Y^1)&Z) | ((Y)&(Z^1));
207     byte C = ((Y^1)&(Z^1)) | ((Y)&(Z^1));
208
209     byte ledOut = C | (B<<1) | (A<<2);
210     ledOut = ledOut << 2;
211     return ledOut;
212 }
213
214 byte incTenHr(byte led1){
215     byte led = led1 >> 2;
216
217     byte Y = led & 1;
218     byte X = (led & (1<<1))>>1;
219
220     byte A = Y;
221     byte B = (X^1)&(Y^1);
222
223     byte ledOut = B | (A<<1);
224     ledOut = ledOut << 2;

```

```

225 return ledOut;
226
227 }

```

The above code runs the clock. Like said before, we use the concept of multiplexing. By that, I mean, I loop through each 7-segment display and power them for a short enough time (2ms). This will make it look like all the six of them are powered at the same time.

I have made three separate functions that increment hours, minutes and seconds separately and a nested if-else to join them all together.

4 USER INTERFACE

The first mode is normal function mode, where it displays currently set time(normal mode). In mode two, we can set time being displayed(set mode).To switch between the modes, we press S1.

Once you enter the set mode, the seconds segment will start blinking. This indicates currently seconds is selected to change. To circulate between seconds, minutes and hours segments, press S. The currently selected segment will blink. To change the time displayed the current segment, press S3. For hours and minutes, the time increments, meanwhile for seconds, it resets back to 00. The clock doesn't stop ticking in this mode.

In normal mode, the clock runs normally and buttons S2 and S3 are disabled.